



# Caching Strategies

In this lesson, we will discuss some of the commonly used caching strategies.

We'll cover the following



- Cache aside
- Read-through
- Write-through
- Write-back

Here are some of the commonly used caching strategies in application development: *cache aside*, *read-through*, *write-through* and *write-back*. Every caching strategy has a specific use case.

Let's find out what they are.

## Cache aside

Cache aside is the most commonly used caching strategy. In this approach, the cache works along with the database intending to reduce the hits on it as much as possible.

The data is lazy-loaded in the cache. When the user sends a request for particular data, the system first looks for it in the cache. If present, it is simply returned. This is called a cache hit.



If not, this is called a cache miss. In this case, the application fetches the data from the database and returns it to the user, also updating the cache for



future requests.

When it comes to data write, it is directly written to the database. Now, this could cause data inconsistency between the cache and the database. To avoid this, the data on the cache has a TTL (Time to live). After it expires, the data is invalidated from the cache.

This caching strategy works best with read-heavy workloads. The data that does not get updated too frequently, like customer data (name, account number, etc.) is cached using the cache aside strategy. We can assign a long TTL to this kind of data.

## Read-through

This strategy is similar to the cache aside strategy. A subtle difference is that the cache in a read-through strategy always automatically stays consistent



The cache library, or the framework, takes the onus of maintaining consistency with the database. On the contrary, in the cache aside strategy, we have to write explicit logic to update the cache.

Application data in this strategy is also lazy-loaded in the cache only when the user requests it. Also, the data model of the cache has to be consistent with the database since it is updated automatically by the library.

## Write-through

In this strategy, the cache sits in line with the database. Every write goes through the cache before updating the database.

This maintains high data consistency between the cache and the database. However, it adds a little latency during the write operations since the data has to be additionally written to the cache. Well, this is a trade-off.



This caching strategy works well for use cases where we need strict data consistency between the cache and the database. It is generally used with other caching strategies to achieve optimized performance.



## Write-back

This caching strategy helps optimize application hosting costs significantly. In it, the data is directly written to the cache instead of the database, and the cache, after some delay, as per the business logic, writes data to the database.

This is what I pulled off in my stock market game. If there are quite a number of writes in the application, developers can reduce the frequency of database writes to cut down the load on it and the associated write operation costs.

A risk in this approach is if the cache fails before the DB is updated, the data might get lost. Again, this strategy is clubbed with other caching strategies to get the best of all the worlds.

Folks, with this, we are done with the caching mechanism in web applications. Now let's move on to the world of message queues.

[← Back](#)[☑ Mark As Completed](#)[Next →](#)[Reducing the Application Deployment ...](#)[Caching Quiz](#)